QA Consulting Pre Assessment Day Exercise 1

Encapsulation

Encapsulation is the practice of grouping variables and methods acting on those variables in a single class. This allows access to the variables to be limited to just the methods in the class (e.g. via the private keyword in Java). This is desirable as it makes the code easier to maintain through allowing the developer to change the implementation of the class without affecting other classes. In addition, encapsulation serves to reduce complexity in software systems by supporting abstraction through giving outside users of a class a streamlined way of using it without having to know exactly how it is implemented.

To draw a real world analogy, encapsulation can be demonstrated in the relationship a driver has with a car. Although the car has an engine the driver never interacts directly with it. Instead they press the accelerator which then causes the engine to accelerate. The driver does not need to know how the engine works and the engine can be completely replaced without changing the behaviour of the accelerator.

The IntegerSet class in Figure 1 demonstrates how encapsulation can be achieved in Java and why it is desirable. The goal of the IntegerSet is to store a unique set of integers. This can be done by having an addInteger method that adds an integer to a member variable list only if it is not already in the list. If the list were publically accessible then other parts of the program could add to the list without verifying that it does not already contain the integer. Furthermore, the developer can change the type of variable holding the integers to a HashSet for improved performance (e.g. for constant access speed when checking if an Integer is already in the set) without affecting how other parts of the program interface with the class. This can be seen in Figure 2 below.

Figure 1: IntegerSet class

```java
import java.util.ArrayList;

public class IntegerSet {

    private ArrayList<Integer> intSet;

    public IntegerSet() {
        intSet = new ArrayList<>();
    }

    public void addInteger(int intToAdd){
        for(int i : intSet){
            if(i == intToAdd) {
                return;
            }
        }
        intSet.add(intToAdd);
    }

}
```

Figure 2: Updated IntegerSet

```java
import java.util.HashSet;

public class IntegerSet {

    private HashSet<Integer> intSet;

    public IntegerSet() {
        intSet = new HashSet<>();
    }

    public void addInteger(int intToAdd){
        intSet.add(intToAdd);
    }
}
```

Inheritance

Inheritance allows classes to obtain the methods and member variables of another class. In this relationship the inheriting class is known as the subclass and the class providing the properties and methods for inheritance is referred to as the superclass. One of the primary benefits of inheritance is that it avoids code duplication as commonalities amongst various classes can be abstracted into a single parent class meaning that the subclasses only have to model the behaviour that varies between them. If a change to the code is required then the places this needs to be made and tested will be limited.

In the real world, inheritance can be demonstrated in a manufacturing environment. A company may produce a base model of a car which has a particular frame and styling. There may then be various models derived from this such a sport edition which adds a higher performance engine and replaces the stock wheels.

Inheritance is demonstrated below in the relationship between the MobilePhone class (Figure 3) and the IPhoneX (Figure 4) and Nokia3310 (Figure 5) classes. The Main class (Figure 6) shows how both iphone and nokia objects can utilise common behaviour outlined in the MobilePhone class though retrieving the phone number associated with the handset and making a call. The console output of this can be seen in (Figure 7). If the developer needed to change how the makeCall method is implemented this can be done in the MobilePhone class and will automatically apply to the subclasses, thus making the code easier to maintain. Inheritance allows each subclass to focus on areas unique to them. For example, the IPhoneX subclass adds an appleId field along with functionality to make FaceTime calls. In addition, inheritance can be flexible due to the ability to override behaviours that do not apply to the subclass. For example the Nokia3310 class overrides the connectToMobileData method as it does not have that functionality. This avoids the developer having to create a duplicate MobilePhone class without a connectToMobileData method that accounts for older phones.

Figure 3: MobilePhone parent class

```java
public abstract class MobilePhone {

    private String myPhoneNumber;

    public MobilePhone(String phoneNumber){
        myPhoneNumber = phoneNumber;
    }

    public String getMyNumber(){
        return myPhoneNumber;
    }

    public void makeCall(String phoneNumber){
        System.out.println("Making call to " + phoneNumber);
    }

    public void connectToMobileData(){
        System.out.println("Establishing to mobile data connection");
    }

}
```

Figure 4: IPhoneX subclass

```java
public class IPhoneX extends MobilePhone{

    private String appleId;

    public IPhoneX(String myNumber, String appleId){
        super(myNumber);
        this.appleId = appleId;
    }

    public String getAppleId(){
        return appleId;
    }

    public void makeFaceTimeCall(String phoneNumber){
        System.out.println("Making FaceTime call to: " + phoneNumber);
    }
}
```

Figure 5: Nokia3310 subclass

```java
public class Nokia3310 extends MobilePhone{

    public Nokia3310(String myNumber){
        super(myNumber);
    }

    public void PlaySnake(){
        System.out.println("Playing Snake!");
    }

    @Override
    public void connectToMobileData(){
        System.out.println("Feature not available");
        System.out.println("Wouldn't you rather play Snake?");
    }
}
```

Figure 6: Inheritance Main class

```java
public class Main {
    public static void main(String[] args) {
        IPhoneX iphone = new IPhoneX( myNumber: "01234 567890",  appleId: "appleID@email.com");
        Nokia3310 nokia = new Nokia3310( myNumber: "98765 432109");

        System.out.println(iphone.getMyNumber());
        System.out.println(nokia.getMyNumber());

        iphone.makeCall( phoneNumber: "98765 432109");
        nokia.makeCall( phoneNumber: "01234 567890");

        System.out.println(iphone.getAppleId());
        iphone.makeFaceTimeCall( phoneNumber: "98765 432109");

        iphone.connectToMobileData();
        nokia.connectToMobileData();
    }

}
```

Figure 7: Inheritance Console output

```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
01234 567890
98765 432109
Making call to 98765 432109
Making call to 01234 567890
appleID@email.com
Making FaceTime call to: 98765 432109
Establishing to mobile data connection
Feature not available
Wouldn't you rather play Snake?

Process finished with exit code 0
```

Polymorphism

The term polymorphism is derived from the Greek *poly* which means many and *morphos* which means forms. In the context of Object Oriented Programming this means that an object can be held under different types of references depending on the classes that it inherits from or the interfaces it implements. The benefit of this is that it can allow a developer to write a single method that takes a more general parent class/interface type as input instead of having to write a separate method for each subclass to perform a method described in the parent class/interface. Although using a more general class/interface limits the methods accessible (unless casting is used), if a method being called is overridden in the subclass this implementation will be invoked.

Polymorphsim can be demonstrated using animals as an analogy. If I had a dog named Spot I could say that Spot is a Dog, a Pet and an Animal. Figures 8, 9 and 10 model this relationship in Java. As can be seen in the Main class (Figure 12) the Dog object can be passed to the eatDinner method which accepts an Animal as its parameter. Similarly, a Tiger object can also be passed to the method. The eatDinner method can then access any methods defined in the Animal class for each animal. As can be seen the implementation of makeSound in each subclass is called as makeSound is an abstract method in the parent (this forces the subclass to provide an implementation). Similarly, the eat behaviour for the Tiger object is retrieved from the Tiger subclass despite the object reference being for an Animal and Animal providing an implementation. As the Dog class does not override the eat method from Animal the Dog object calls the eat method as defined in Animal. By using polymorphism the developer does not need utilise overloading by creating 2 eatDinner methods, one for a Dog and one for a Tiger. Instead the developer creates one method which still has the power to access the overrides a class makes and can work with any new subclasses that are created under Animal. Polymorphism can also prove useful here by allowing the developer to hold both the Tiger and Dog objects in a single collection of Animal references, such as a List. The Main class also shows how an Interface can used to achieve polymorphism by forcing the Dog class to implement a play method which can then be called through a Pet reference.

Figure 8: Animal parent class

```
public abstract class Animal {
    public abstract void makeSound();

    public void eat(){
        System.out.println("Eating food!");
    }
}
```

Figure 9: Pet interface

```
public interface Pet {
    public void play();
}
```

Figure 10: Dog subclass

```java
public class Dog extends Animal implements Pet{

    private String name;

    public Dog(String name){
        this.name = name;
    }

    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }

    @Override
    public void play() {
        System.out.println(name + " rolls over!");
    }
}
```

Figure 11: Tiger subclass

```java
public class Tiger extends Animal{
    @Override
    public void makeSound() {
        System.out.println("ROAR!!!");
    }

    @Override
    public void eat(){
        System.out.println("Devouring Food!");
    }
}
```

Figure 12: Polymorphism Main class

```java
public class Main {

    public static void main(String[] args) {

        Tiger t = new Tiger();
        Dog d = new Dog( name: "Spot");

        eatDinner(t);
        eatDinner(d);

        playWithPet(d);
    }

    public static void eatDinner(Animal a){
        a.makeSound();
        a.eat();
    }

    public static void playWithPet(Pet p){
        p.play();
    }
}
```

Figure 13: Polymorphism Console output

```
"C:\Program Files\Java\jdk1.8.0_111\bin\java" ...
ROAR!!!
Devouring Food!
Woof!
Eating food!
Spot rolls over!

Process finished with exit code 0
```

Abstraction

Abstraction is the concept of hiding unnecessary details from a user and focusing on providing essential information regarding the behaviour of an object instead of how it was implemented. In day to day life abstractions are used in order to communicate concepts more efficiently and streamline how people interface with objects. For example, someone can type a desired letter on a keyboard without having to know how a specific letter is read by the computer and displayed on screen.

One way Abstraction can be achieved in java is through Abstract classes. These classes are declared using the abstract keyword and cannot be instantiated. Figure 14 shows the abstract Animal class discussed earlier. It would not make sense to instantiate an Animal as any animal is inherently its own type of animal. The Animal class abstracts features common to Animals such as making sound and eating. By providing an implementation for the eat method code duplication is reduced as subclasses can inherit this behaviour. By using the abstract keyword in the make sound method the Animal class forces any class extending it to provide and implementation of the make sound method (Figure 15). A user using data of the Animal type knows that it will have a method named makeSound which consumes no parameters and returns no value. They do not need to know how the method is specifically implemented in the subclass but can call it and assume that it will make the Animal object make some form of noise. Abstraction can also be attained through the use of interfaces in Java. Classes can implement multiple interfaces using the implements keyword. These effectively act as contract which outline method signatures which the implementing class must implement. Figure 16 shows the Pet interface that Dog implements. By forcing the Dog subclass to provide the implementation of the Pet interface along with the abstract makeSound method from the Animal class complexity is reduced in the interface and abstract class giving the user a more streamlined view of the system.

Figure 14: Abstraction Animal class

```
public abstract class Animal {
    public abstract void makeSound();

    public void eat(){
        System.out.println("Eating food!");
    }
}
```

Figure 15: Abstraction Dog class

```java
public class Dog extends Animal implements Pet{

    private String name;

    public Dog(String name){
        this.name = name;
    }

    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }

    @Override
    public void play() {
        System.out.println(name + " rolls over!");
    }
}
```

Figure 16: Abstraction Interface

```java
public interface Pet {
    public void play();
}
```